

DISEÑO DE BASE DE DATOS

Concesionario CampusCar

Modelo Entidad-Relación, Diccionario de Datos y Reglas de Negocio

Frayden Stiven Garcia Urbina

CAMPUSLAND

Julio de 2026

1. Introducción

Este documento presenta el diseño de una base de datos relacional para el sistema de gestión de un concesionario de vehículos. El modelo cubre cuatro procesos centrales del negocio: el control del inventario de vehículos, la administración de clientes y vendedores, el registro de transacciones de venta y la gestión de servicios de mantenimiento, ya sea antes o después de la venta del vehículo.

El diseño se realizó siguiendo la metodología Entidad-Relación (E-R), partiendo de los requerimientos funcionales entregados, para luego normalizar el modelo hasta la Tercera Forma Normal (3FN) y traducirlo a un esquema relacional con sus respectivas claves, restricciones y disparadores (triggers) que automatizan reglas de negocio críticas, como la actualización de disponibilidad de un vehículo al venderse.

2. Justificación del Diseño

El modelo se compone de seis entidades: CLIENTE, VENDEDOR, VEHICULO, VENTA, DETALLE_VENTA y MANTENIMIENTO. A continuación se justifican las decisiones más relevantes:

- Entidad asociativa DETALLE_VENTA: el requerimiento indica que "cada venta está asociada a un cliente, un vendedor y uno o más vehículos". Esto implica una relación muchos-a-muchos entre VENTA y VEHICULO (una venta puede incluir varios vehículos, por ejemplo, un vehículo y una motocicleta de cortesía, o un lote de vehículos para flotas). En un modelo relacional, una relación N:M no puede representarse con una sola clave foránea; se requiere una tabla intermedia. Por eso se creó VENTA_VEHICULO, con clave primaria compuesta (id_venta, vin) y el atributo precio_venta, que registra el precio real pactado por unidad dentro de esa transacción (puede diferir del precio de lista en VEHICULO).
- Separación de VENTA (encabezado) y VENTA_VEHICULO (detalle): esta es la misma lógica de una factura con líneas de detalle. VENTA guarda los datos que aplican a toda la transacción (fecha, total, método de pago, cliente, vendedor) y VENTA_VEHICULO guarda el detalle por vehículo. Esto evita duplicar fecha/cliente/vendedor por cada vehículo vendido en una misma transacción y facilita el cálculo de totales.
- MANTENIMIENTO con cliente opcional: el requerimiento indica explícitamente que los vehículos "pueden recibir mantenimiento (con clientes o sin haber sido vendidos aún)". Por eso id_cliente en MANTENIMIENTO es NULLABLE: un mantenimiento de pre-entrega sobre un vehículo nuevo en inventario no tiene cliente asociado, mientras que un mantenimiento correctivo posventa sí lo tiene.
- VIN como clave primaria natural de VEHICULO: el número de serie (VIN) es único por definición en la industria automotriz (17 caracteres alfanuméricos bajo la norma ISO 3779). En lugar de crear un id_vehiculo artificial, se usó el VIN como clave primaria natural, reforzado además con la restricción UNIQUE/PRIMARY KEY a nivel de motor de base de datos.
- Atributo disponible en VEHICULO: en vez de inferir la disponibilidad consultando si el vehículo tiene una venta asociada (lo cual sería costoso y ambiguo ante devoluciones o anulaciones), se mantiene una bandera booleana disponible que se actualiza automáticamente mediante triggers, garantizando que el inventario siempre refleje el estado real sin depender de la lógica de la aplicación cliente.
- Vendedores y ventas: cada venta requiere exactamente un vendedor responsable (relación 1:N obligatoria), lo que permite calcular comisiones y desempeño por vendedor.

3. Diagrama Entidad-Relación (UML)

El siguiente diagrama resume las entidades, atributos y relaciones del modelo, con notación de cardinalidad tipo "pata de gallo" (crow's foot). El archivo fuente editable se encuentra en el repositorio como diagrama_er.mmd (formato Mermaid, con renderizado nativo en GitHub) y diagrama_er.dot / diagrama_er.svg (Graphviz).

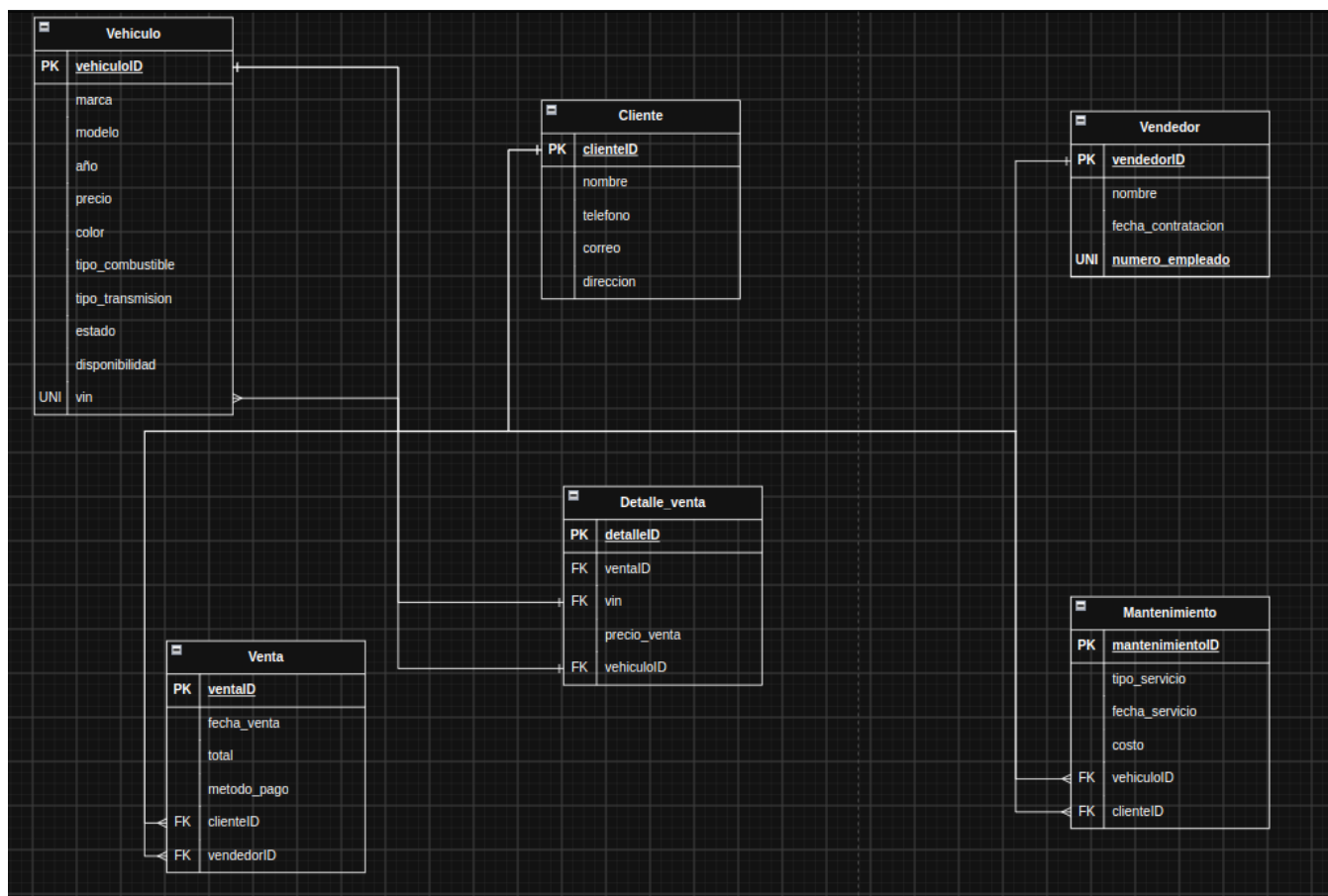


Figura 1. Modelo Entidad-Relación del sistema de concesionario de vehículos.

4. Diccionario de Datos

4.1 CLIENTE

Representa a las personas que compran vehículos y/o solicitan servicios de mantenimiento.

Atributo	Tipo	Restricción / Descripción
id_cliente	INT (SERIAL)	Clave primaria (PK), autoincremental
nombre	VARCHAR(150)	NOT NULL
teléfono	VARCHAR(20)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE — evita registros duplicados de un mismo cliente
dirección	VARCHAR(250)	Opcional

4.2 VENDEDOR

Representa al personal comercial encargado de gestionar las ventas.

Atributo	Tipo	Restricción / Descripción
id_vendedor	INT (SERIAL)	Clave primaria (PK)
nombre	VARCHAR(150)	NOT NULL
numero_empleado	VARCHAR(20)	NOT NULL, UNIQUE — identificador interno de RR.HH.
fecha_contratación	DATE	NOT NULL

4.3 VEHICULO

Representa cada unidad física en el inventario del concesionario, nueva o usada.

Atributo	Tipo	Restricción / Descripción
vin	VARCHAR(17)	Clave primaria (PK) — clave natural única mundialmente
marca	VARCHAR(80)	NOT NULL
modelo	VARCHAR(80)	NOT NULL
año	SMALLINT	NOT NULL, CHECK entre 1900 y 2100
precio	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 (precio de lista)
color	VARCHAR(40)	Opcional
tipo_combustible	VARCHAR(20)	NOT NULL, CHECK IN (Gasolina, Diesel, Hibrido, Electrico, Gas)
tipo_transmisión	VARCHAR(20)	NOT NULL, CHECK IN (Manual, Automatica, CVT)
estado	VARCHAR(10)	NOT NULL, CHECK IN (Nuevo, Usado)
disponible	BOOLEAN	NOT NULL, DEFAULT TRUE — actualizado por trigger al vender

4.4 VENTA

Encabezado de la transacción comercial.

Atributo	Tipo	Restricción / Descripción
id_venta	INT (SERIAL)	Clave primaria (PK)
fecha_venta	DATE	NOT NULL, DEFAULT fecha actual
total	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0
método_pago	VARCHAR(20)	NOT NULL, CHECK IN (Efectivo, Tarjeta, Transferencia, Financiamiento)
id_cliente	INT	Clave foránea (FK) → cliente(id_cliente), NOT NULL
id_vendedor	INT	Clave foránea (FK) → vendedor(id_vendedor), NOT NULL

4.5 VENTA_VEHICULO

Entidad asociativa (tabla de detalle) que resuelve la relación muchos-a-muchos entre VENTA y VEHICULO.

Atributo	Tipo	Restricción / Descripción
id_venta	INT	Clave primaria compuesta y FK → venta(id_venta)
vin	VARCHAR(17)	Clave primaria compuesta y FK → vehiculo(vin)
precio_venta	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 — precio pactado en esa venta

4.6 MANTENIMIENTO

Registra cada servicio técnico realizado a un vehículo, haya sido vendido o no.

Atributo	Tipo	Restricción / Descripción
id_mantenimiento	INT (SERIAL)	Clave primaria (PK)
tipo_servicio	VARCHAR(30)	NOT NULL, CHECK IN (Preventivo, Correctivo, Garantía, Pre-entrega)
costo	NUMERIC(10,2)	NOT NULL, CHECK ≥ 0
fecha_servicio	DATE	NOT NULL, DEFAULT fecha actual
vin	VARCHAR(17)	Clave foránea (FK) → vehiculo(vin), NOT NULL
id_cliente	INT	Clave foránea (FK) → cliente(id_cliente), NULLABLE

5. Relaciones UML y Cardinalidades

Atributo	Tipo	Restricción / Descripción
CLIENTE → VENTA	1 : N	Un cliente puede realizar varias ventas (o ninguna); toda venta pertenece exactamente a un cliente. Relación obligatoria del lado de VENTA (id_cliente NOT NULL).
VENDEDOR → VENTA	1 : N	Un vendedor gestiona varias ventas; toda venta tiene exactamente un vendedor responsable. Relación obligatoria (id_vendedor NOT NULL).
VENTA → VENTA_VEHICULO	1 : N	Una venta incluye uno o más vehículos (al menos uno, por regla de negocio aplicada en la capa de aplicación / transacción).
VEHICULO → VENTA_VEHICULO	1 : N	Un vehículo aparece, como máximo, en una venta (regla de negocio reforzada por trigger: no se puede insertar un vehículo ya vendido en una nueva venta).
VEHICULO → MANTENIMIENTO	1 : N	Un vehículo puede tener cero o varios servicios de mantenimiento a lo largo de su vida útil (antes y después de la venta).
CLIENTE → MANTENIMIENTO	0..1 : N	Relación opcional: un mantenimiento puede o no estar asociado a un cliente, según si el vehículo ya fue vendido.

En conjunto, VENTA y VEHICULO se relacionan de forma muchos-a-muchos (N:M) a través de la entidad asociativa VENTA_VEHICULO, la cual se descompone en dos relaciones 1:N como se muestra en la tabla anterior — este es el mecanismo estándar para representar relaciones N:M en un modelo relacional.

6. Restricciones y Validaciones

6.1 Claves primarias (PK)

- cliente.id_cliente, vendedor.id_vendedor, venta.id_venta, mantenimiento.id_mantenimiento: claves subrogadas autoincrementales (SERIAL).
- vehiculo.vin: clave primaria natural (17 caracteres), garantiza unicidad del vehículo en todo el sistema.
- venta_vehiculo (id_venta, vin): clave primaria compuesta — impide registrar el mismo vehículo dos veces dentro de la misma venta.

6.2 Claves foráneas (FK) e integridad referencial

- venta.id_cliente → cliente.id_cliente (ON DELETE RESTRICT): no se permite borrar un cliente con ventas registradas, preservando el historial.
- venta.id_vendedor → vendedor.id_vendedor (ON DELETE RESTRICT): mismo criterio para preservar el historial de ventas por vendedor.
- venta_vehiculo.id_venta → venta.id_venta (ON DELETE CASCADE): si se anula/elimina una venta completa, se eliminan sus líneas de detalle (y el trigger asociado libera el vehículo).
- venta_vehiculo.vin → vehiculo.vin (ON DELETE RESTRICT): no se puede eliminar un vehículo que ya está asociado a una venta.

- mantenimiento.vin → vehiculo.vin (ON DELETE RESTRICT): protege el historial de mantenimiento del vehículo.
- mantenimiento.id_cliente → cliente.id_cliente (ON DELETE SET NULL): si se elimina un cliente, sus registros de mantenimiento permanecen para trazabilidad del vehículo, pero pierden el vínculo con el cliente.

6.3 Restricciones de unicidad (UNIQUE)

- vehiculo.vin: unicidad garantizada por ser PK; adicionalmente se valida el formato de 17 caracteres a nivel de aplicación.
- cliente.email: evita registrar el mismo cliente más de una vez con datos distintos.
- vendedor.numero_empleado: evita duplicidad del identificador interno del empleado.

6.4 Restricciones de dominio (CHECK)

- vehiculo.estado: solo admite 'Nuevo' o 'Usado'.
- vehiculo.tipo_combustible y tipo_transmision: catálogos cerrados de valores válidos.
- vehiculo.anio: rango razonable (1900–2100).
- venta.metodo_pago: solo admite 'Efectivo', 'Tarjeta', 'Transferencia' o 'Financiamiento'.
- Todos los campos monetarios (precio, total, costo, precio_venta): CHECK >= 0.

6.5 Reglas de negocio automatizadas (triggers)

Para cumplir el requerimiento "un vehículo vendido debe cambiar su estado a 'no disponible'" sin depender de que la aplicación lo actualice manualmente (lo que podría olvidarse o hacerse de forma inconsistente), se implementaron disparadores a nivel de base de datos:

- trg_marcar_vehiculo_vendido (AFTER INSERT en venta_vehiculo): al registrar un vehículo en una venta, el trigger actualiza automáticamente vehiculo.disponible = FALSE. Además, valida que el vehículo no haya sido vendido previamente, lanzando una excepción si ya está marcado como no disponible — esto refuerza a nivel de motor la regla de que un vehículo solo puede venderse una vez.
- trg_liberar_vehiculo (AFTER DELETE en venta_vehiculo): si se anula una venta, el vehículo regresa automáticamente a disponible = TRUE.
- vw_control_totales_venta: vista de apoyo para auditoría, que compara el total registrado en VENTA contra la suma real de los precios de los vehículos en VENTA_VEHICULO.

El detalle completo de la implementación (DDL, CHECK, triggers y datos de prueba) se encuentra en el archivo schema.sql del repositorio.

7. Conclusiones

El modelo propuesto cumple con todos los requerimientos funcionales solicitados: gestión de inventario de vehículos con disponibilidad siempre actualizada, registro de clientes y vendedores, transacciones de venta que pueden incluir múltiples vehículos, y servicios de mantenimiento asociables o no a un cliente. El uso de una entidad asociativa para resolver la relación N:M entre ventas y vehículos, junto con triggers que automatizan la actualización de disponibilidad, garantiza consistencia de datos independientemente de la aplicación cliente que consuma la base de datos.

Como trabajo futuro, el modelo puede extenderse con entidades adicionales como SUCURSAL (si el concesionario tiene varias sedes), PROVEEDOR (para el origen de los vehículos) o GARANTIA (para políticas de cobertura por marca/modelo).